

Issue #003 - Overlapping matches corrupt Themis output

Date: 2026-07-19

Status: Root cause confirmed - handover to Themis engineering

Category: Themis Runtime Defect / Data Correctness

Severity: High - silent, unbounded data loss

Owner: Themis engineering

Reported by: Scale Validation Qualification

HANDOVER SUMMARY

Root cause

When two rules in a policy match **overlapping regions of the input**, the Themis runtime computes the wrong start offset for the replacement and overwrites content preceding the match.

Either rule alone produces correct output. Only their coexistence triggers the defect, and the order in which they appear in the policy does not matter.

Overlap is the trigger. Adjacency is safe.

case	input	correct	actual
disjoint matches	a X b Y c	a [P] b [Q] c	correct
adjacent, touching but not overlapping	x AAAABBBB y	x [P][Q] y	correct
overlap by 1 character	x ABCDEFG y	x [P]EFG y	x [P[Q] y
overlap by 3 characters	x ABCDEFGHI y	x [P]GHI y	x [Q] y

Rules: "ABCD" -> "[P]", "DEFG" -> "[Q]" and "ABCDEF" -> "[P]", "DEFGHI" -> "[Q]".

In the 3-character overlap case the runtime destroyed AB - legitimate input that neither rule matched.

Note on an earlier, narrower framing

This issue previously described the trigger as "one rule's literal is a strict prefix of another rule's literal". That is a real trigger but only a special case: a prefix literal always produces a totally overlapping match.

The general condition is overlapping matches. Two literals can produce overlapping matches without either containing the other - "ABCD" and "DEFG" share no containment relationship, yet corrupt output.

Statically, two literals A and B can produce overlapping matches when either contains the other, or when some non-empty proper suffix of A equals a proper prefix of B. This is checkable without knowing the input.

Matching semantics (established while characterising this)

- Replacement output is NOT re-scanned. Rules "SEED" -> "GROWN" and "GROWN" -> "[RESCANNED]" applied to value SEED here yield value GROWN here. Single pass, confirmed.
- Where matches do not overlap, results are correct and order-independent.

Minimal reproduction

Two rules, one record. No scale required.

Policy:

```
"Elena Chen 1327" -> "[PII:PERSON_NAME]";
"Elena Chen"      -> "[PII:PERSON_NAME]";
```

Input:

```
name: Elena Chen 1327, done
```

Observed output:

```
name: [PII:[PII:PERSON_NAM, done
```

Control cases, same input:

policy	output
full rule only	name: [PII:PERSON_NAM, done
prefix rule only	name: [PII:PERSON_NAM 1327, done
both rules, prefix listed second	name: [PII:[PII:PERSON_NAM, done
both rules, prefix listed first	name: [PII:[PII:PERSON_NAM, done
both rules, replacement [NAME]	na [NAME], done
both rules, replacements [FULL]/[PFX]	n [FULL], done

Reproducing without any of our tooling

If you are reading this outside the validation framework repository, the defect reproduces with curl alone against your own tenant. Nothing to install.

The policy load REPLACES the active policy, so use a tenant where that is acceptable.

```

POLICY_ENDPOINT="https://<host>:8444/policy"
PROCESS_ENDPOINT="https://<tenant>/v1/process"
TOKEN="<token>"

# 1. Load two rules whose matches overlap
printf '"ABCD" -> "[P]";\n"DEFG" -> "[Q]";\n' | curl -sS \
  -X POST "$POLICY_ENDPOINT" \
  -H "Authorization: Bearer $TOKEN" --data-binary @-

# 2. Send one record containing ABCDEFG
curl -sS -X POST "$PROCESS_ENDPOINT" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"message":"x ABCDEFG y"}'
#   correct:  x [P]EFG y
#   observed: x [P[Q] y

# 3. Control - load either rule alone and repeat step 2
printf '"ABCD" -> "[P]";\n' | curl -sS \
  -X POST "$POLICY_ENDPOINT" \
  -H "Authorization: Bearer $TOKEN" --data-binary @-
#   returns x [P]EFG y correctly

```

Add `-k` if the endpoint presents a self-signed certificate.

Scripted reproductions (validation framework repository only)

`scripts/repro-issue-003-curl.sh` and `scripts/repro-issue-003.py` cover the full case matrix including controls. These belong to the validation framework and are not needed to reproduce the defect.

Why it matters

The corruption is silent. Every request returns HTTP 200. Nothing in the response indicates the output is wrong.

The data loss is unbounded. It is not limited to a delimiter. In the `[NAME]` case above, four characters of legitimate preceding content (`me:`) were destroyed; in the `[FULL]` case, five (`ame:`). The number of destroyed bytes varies with the configured replacement lengths, so a policy with short replacements loses MORE surrounding data, not less.

In CSV output this merges fields and produces structurally invalid records:

```

expected:  CUST-647637,[PII:PERSON_NAM
actual:    CUST-647637[PII:PERSON_NAM

```

Evidence

1. Bisecting the catalog isolates a single causal rule

Policies were built as prefixes of the qualification catalog, preserving rule content, ordering, and position.
Probe literal: **Caroline Ramirez 1291**.

```
1,699 rules -> clean
1,700 rules -> corrupt
```

Rule 1,700 is:

```
"Caroline Ramirez" -> "[PII:PERSON_NAME]";
```

a strict prefix of the probe literal.

2. Removing prefix rules eliminates the corruption

The full catalog contains 31 literals that are a strict prefix of another literal. Removing only those rules (5,000 -> 4,969):

literal	full catalog	prefix rules removed
Caroline Ramirez 1291	CORRUPT -1	clean
Alicia Johnson 3763	CORRUPT +1	clean
Alicia Wright 1543	CORRUPT +2	clean
Alicia Patel 1243	CORRUPT +3	clean
Elena Chen 1327	CORRUPT +5	clean
Alicia Clark 3043 (ctl)	clean	clean
Alicia Chen 1723 (ctl)	clean	clean

Every affected literal has exactly one prefix rule in the catalog. Neither control literal has one.

3. Among containment classes, only prefix corrupts

Read this together with the handover summary. Containment is not the only trigger - non-containment overlaps such as **"ABCD"** with **"DEFG"** also corrupt. This section narrows the containment case specifically; it does not establish that non-containment literals are safe.

Three containment classes were tested against Themis with curl:

policy	input	output
"Elena Chen 1327", "Elena Chen" (prefix)	name: Elena Chen 1327, done	na[NAME], done CORRUPT
"3104 Cedar Avenue", "104 Cedar Avenue" (suffix)	addr: 3104 Cedar Avenue, done	addr: [ADDR], done correct

policy	input	output
"XX Elena Chen YY", "Elena Chen" (middle)	name: XX Elena Chen YY, done	name: [MID], done correct
"Elena Chen 1327", "Robert Smith 9999" (disjoint)	name: Elena Chen 1327, done	name: [NAME], done correct

Within containment, only a strict PREFIX corrupts. Suffix and middle containment render correctly, because in those cases the two matches begin at different offsets and the runtime resolves them without displacing the start.

The unifying condition remains overlapping matches. A prefix literal produces a totally overlapping match; suffix and middle containment in these tests did not produce overlapping selected matches.

4. Replacement length is not a factor

A 7-rule policy containing no prefix overlaps was tested with replacements of 17, 15, and 10 characters. All produced correct output for all literals, including every literal that fails at full catalog.

Scale is not the trigger

Catalog size correlates with the defect only because a larger generated catalog is more likely to contain an overlapping pair. The 5,000-rule qualification and the 2-rule reproduction exhibit identical behavior.

Observed offset arithmetic

For the qualification workload, where the replacement rendered as a 15-character token, the corrupted region always spanned 20 bytes regardless of matched literal length `L`, giving:

delta = len(actual) - len(expected) = 20 - L

L	15	16	17	18	19	20	21
delta	+5	+4	+3	+2	+1	0	-1
count	38	53	59	45	35	-	8

This law was derived from the 272 qualification failures and subsequently predicted the delta for all five probe literals correctly in isolated single-record tests.

The constant is not universal. It varies with the configured replacement lengths, as the `[NAME]` and `[FULL]/[PFX]` cases above demonstrate. The invariant is that the match end is correct and the start is displaced.

Relationship to KB-001

KB-001 (replacement strings truncated to 15 characters) is a separate and unrelated behavior. It is reproducible with a single rule and no overlap; this defect is reproducible with a short replacement and no

truncation.

An earlier revision of this issue suggested the two might share a root cause. That is now disproved - see "Replacement length is not a factor" above.

Constraining replacements to 15 characters does NOT prevent this corruption.

Suggested area to investigate

Match start offset computation when more than one rule matches at the same position. The end offset is correct in every observed case, so the fault is likely in deriving start from end using a length belonging to the wrong matched rule.

INVESTIGATION RECORD

The material below is the working record, retained for provenance. It contains hypotheses that were subsequently disproved; the handover summary above is authoritative.

Superseded readings, for the record:

- "Truncation explains the failures" - disproved. Failures persist under normalization at 15 characters.
- "Replacement rescanning" - disproved. Rescanning would produce a well-formed nested token; the observed fragments are variable-length partial prefixes, and negative-delta cases consume unrelated preceding characters.
- "Prefix collision is ruled out" - WRONG. Tested against the 395 literals appearing in records rather than the full 5,000-rule catalog, which excluded the prefix rules. Prefix collision is the confirmed root cause.
- "Double-advance against a fixed 20-byte reference" - partially superseded. The 20-byte constant is specific to the qualification's replacement lengths, not intrinsic. The invariant is that the match end is correct and the start is displaced.
- "Catalog scale is required" - superseded. Scale correlates only because a larger generated catalog is more likely to contain an overlapping pair. Two rules suffice.

Context

This issue was discovered during the first large-scale validation qualification intended to determine whether the validation framework could support realistic customer proof-of-concept workloads.

Qualification parameters:

- Rules requested: 5,000
- Records requested: 10,000
- Target: Themis
- Run ID: 20260719T161514709224Z

Observed Behavior

Functional execution completed successfully.

Results:

- Records processed: 10,000
- Requests succeeded: 10,000
- Requests failed: 0

Latency:

- Average latency: 12.576 ms
- p50: 12.498 ms
- p95: 14.207 ms
- p99: 16.462 ms

Validation results:

- Records evaluated: 10,000
- Records passed: 9,728
- Records failed: 272
- Pass rate: 97.280%

Failure classification:

- CONTENT_MISMATCH: 272
- EXECUTION_FAILURE: 0

Expected Behavior

For every dirty record, Themis should replace each detected policy literal with the configured replacement token.

Example:

Input:

```
Elena Chen 2527
```

Expected:

```
[PII:PERSON_NAME]
```

Actual Behavior

Observed output contains malformed replacement results including:

Truncated replacement tokens:

```
[PII:PERSON_NAM  
[PII:EMAIL_ADDR  
[PII:STREET_ADD
```

Nested replacements:

```
[PII: [PII:PERSON_NAM
```

The framework-generated expected output does not contain these patterns.

Evidence Collected

Confirmed:

- Rule catalog generation produced 5,000 unique detection literals.
- Policy generation completed successfully.
- Generated policy artifacts contain complete replacement strings.
- Themis accepted and loaded all 5,000 rules.
- Execution returned HTTP 200 for all 10,000 records.
- Framework expected transformations are deterministic.

A minimal reproduction confirmed the known replacement length limitation:

Run:

```
20260719T174421134465Z
```

Configuration:

```
Rules: 1  
Records: 1
```

Generated policy:

```
"4111-1111-0001-0007" -> "[FINANCIAL:CREDIT_CARD_NUMBER]";
```

Themis returned a truncated replacement.

Comparison behavior:


```
--replacement-max-length 15
```

passes validation.

```
--replacement-max-length 16
```

fails validation.

Related behavior:

```
KB-001 – Replacement strings truncated to 15 characters
```

Root Cause Analysis

Confirmed - replacement truncation (KB-001)

Themis runtime truncates replacement literals longer than 15 characters.

This behavior occurs after policy generation and during runtime transformation processing.

No framework-side policy generation or serialization issue has been identified.

Note: this was initially believed to explain the 272 failures. It does not. See "Confirmed - Source Cursor Misalignment" below.

Confirmed - Source Cursor Misalignment (2026-07-19)

The 272 failures are NOT explained by KB-001.

Re-running comparison with `--replacement-max-length 15` against the existing run output produced an identical result: 272 CONTENT_MISMATCH. The manifest confirms normalization was applied (`replacement_max_length: 15`). The failures survive replacement-length normalization entirely.

All 272 failing records are `dirty` and all involve `[PII:PERSON_NAM`.

Observed corruption is a positional shift of the replacement token, not a malformed replacement value. Two directions occur:

Characters left behind before the token (positive delta):

```
expected: CUST-516774, [PII:PERSON_NAM
actual:    CUST-516774, [PII: [PII:PERSON_NAM
```

Characters consumed before the token (negative delta):

```
expected: CUST-647637, [PII:PERSON_NAM
actual:    CUST-647637[PII:PERSON_NAM
```

The earlier reading of the nested case as "replacement rescanning" was incorrect. Rescanning would produce a well-formed nested token. The observed fragments are variable-length partial prefixes, and the negative cases consume unrelated preceding characters (commas, spaces). Both are consistent with the replacement being written at a misaligned source offset.

Empirical law

Let L = length of the matched `person_name` literal and $\text{delta} = \text{len(actual)} - \text{len(expected)}$:

delta = 20 - L

L	15	16	17	18	19	20	21
delta	+5	+4	+3	+2	+1	0	-1
count	38	53	59	45	35	-	8

Equivalently, the runtime consumes $2L - 20$ source bytes where it should consume L . The error is zero at $L = 20$; no length-20 literal appears in the failure set.

This is the signature of a double-advance: the source cursor is advanced by the matched literal length and then corrected a second time against a fixed 20-byte reference.

Gating factor - RESOLVED

At the time of writing, the gating factor was unknown. Literal length appeared to determine the magnitude of corruption but not whether it occurred:

```
L=17 -> 63 failed, 320 passed
L=15 -> 38 failed, 102 passed
L=19 -> 38 failed, 145 passed
```

The gating factor is the presence of a second rule whose literal is a strict prefix of the matched literal. See the handover summary.

Note on a false negative recorded during this investigation: an earlier analysis tested prefix relationships among the 395 literals that appeared in records and found none, and prefix collision was wrongly ruled out. Prefix rules such as "`Caroline Ramirez`" are present in the catalog but never match a record, so they were absent from the reference set. The hypothesis was tested against the wrong population, not disproved.

Evidence: run `20260719T161514709224Z`, artifact `generated/comparison.jsonl`.

Remaining Investigation

Closed. Root cause confirmed; see handover summary.

Validation-framework follow-up is tracked separately: the scale workload generator emits bare `"First Last"` literals for indices ≤ 400 and `"First Last {index}"` above, so generated catalogs contain overlapping literals by construction.

Impact

This prevents using the current large-scale qualification as customer-facing proof-of-concept evidence until deterministic transformation behavior is confirmed.

The execution path is stable, but transformation correctness must reach 100% before presenting validation results externally.

Recommendation

Themis engineering

Investigate match start offset computation where two matches overlap. See "Reproducing without any of our tooling" above for copy-pasteable curl covering both the containment case and the partial-overlap case.

The end offset is correct in every observed case; the start is displaced. Note that adjacent non-overlapping matches are handled correctly, so the fault is specific to resolving two candidate matches that share input bytes.

Customer guidance, until resolved

No two literals in a policy may produce overlapping matches. Statically:

- Neither literal may contain the other.
- No non-empty proper suffix of one may equal a proper prefix of the other.

Adjacent matches that do not share bytes are safe.

This is a real and awkward authoring constraint. Both classes occur naturally:

- Containment: redacting `"Acme Corp"` and `"Acme Corporation"`.
- Suffix/prefix: redacting `"ACCT-1234"` and `"1234-5678"`, where neither literal contains the other.

Validation framework

Do not change validation expectations to accommodate this behavior. The framework output is correct; masking the corruption would hide a customer-facing data integrity defect.

Two changes are warranted:

- Detect literal pairs that can produce overlapping matches during policy generation, and report them before execution.
- Offer a generation mode that produces no such pairs, so qualification runs can isolate other behavior.

The scale generator currently produces overlapping literals by construction: bare "**First Last**" for indices ≤ 400 and "**First Last {index}**" above. The qualification catalog contained 31 containment pairs; suffix/prefix pairs were not counted and may add more.

Resolution

Root cause confirmed. Awaiting Themis engineering for the runtime fix.

Framework-side: generation, policy deployment, execution, and comparison all behaved correctly and correctly identified the defect. Follow-up work is additive (overlap detection and reporting), not corrective.

Regression Test

Once the runtime defect is resolved, the validation suite should prove:

- 5,000 rules load successfully.
- 10,000 records process successfully.
- Expected transformations equal actual transformations.
- CONTENT_MISMATCH count is zero.
- Results remain deterministic across repeated runs.

Targeted guard for this defect:

- A policy containing a literal and a strict prefix of that literal produces correct output for both.
- Verified with the prefix rule listed first and listed second.
- Verified with replacements shorter than, equal to, and longer than the 15-character KB-001 truncation boundary.
- For every replacement, output length equals input length minus the matched literal length plus the rendered replacement length.
- No character preceding a replacement token is added or removed.

`scripts/repro-issue-003.py` covers these cases and should fail while the runtime defect is present.